

Introduction to Computational Intractability (ADA 2023, CSE 222)

Diptapriyo Majumdar

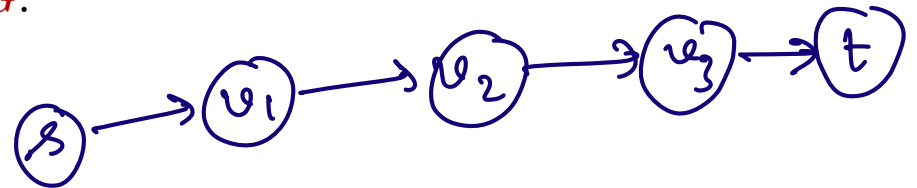
April 13, 2023

Outline

- ① Decision vs Search Problems - Computational Intractability
- ② Another problem in Network Flow

- **Problem-1:** Given a nonnegative edge-weighted directed graph G and two vertices s and t . Design an algorithm that determines if G have a path from s to t with total weight at most k for some $k \in \mathbb{N}$.
- **Problem-2:** Given a nonnegative edge-weighted directed graph G and two vertices s and t . Design an algorithm that determines an explicit path of total shortest weight from s to t in G .

$\text{prev}(v) = u$



Problem 1: Decision problem. Yes/No answer.

Problem 2: Search Problem

- Given a sequence of n weeks, a plan is specified by a choice of 'low-stress job', 'high-stress job', or 'none' for each of the n weeks. If a 'high-stress' job is chosen for $i > 1$, then 'none' should be chosen for the week $i - 1$. But, a high-stress job can be chosen in week 1. However, if a low-stress job can be chosen in a week i , then either of low-stress or high-stress job can be chosen in week $i - 1$. A low stress job gives revenue of a_i and a high-stress job gives a revenue of b_i for week i . Design an algorithm to find a plan for n consecutive weeks that maximizes the revenue.

- An example:**

a_i -values: 14, 7, 24, 10, 18, 8.

b_i -values: 20, 12, 30, 15, 21, 10.

a

14	7	24	10	18	8	
20	12	30	15	21	10	

b

Subproblem: $Rev[k]$ = the maximum possible revenue for the weeks $\{1, 2, \dots, k\}$.

$$Rev[0] = 0 \quad Rev[1] = \max\{a_1, b_1\}.$$

$$Rev[k] = \max\{Rev[k-1] + a_k, Rev[k-2] + b_k\}$$

Revenue

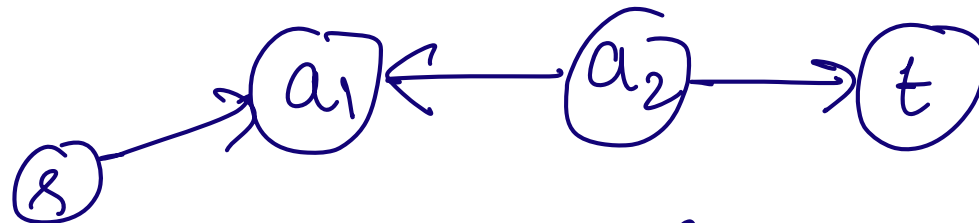
0	20	27	51	61	79	87	
0	1	2	3	4	5	6	
	b_1	a_2	a_3	a_4	a_5	a_6	

87
71

Decision Version: Given the specification
and an integer W , is there a valid
plan with total revenue $\geq W$?

Verification of a problem

- **Problem-1:** Given a nonnegative edge-weighted directed graph G and two vertices s and t . Design an algorithm that determines if G have a path from s to t with total weight at most k for some $k \in \mathbb{N}$.
- **Problem-2:** Given a nonnegative edge-weighted directed graph G and two vertices s and t . Design an algorithm that determines an explicit path of total shortest weight from s to t in G .
- **Question:** How do we define the verification for the same problem?



Given $(G, s, t, \text{ and } S \subseteq V(G), F \subseteq E(G))$
there is a polynomial-time algorithm
that checks if the subgraph

$H(S, F)$ is a path from s to t .

$H(S, F)$ is called a certificate.

- Given a sequence of n weeks, a plan is specified by a choice of 'low-stress job', 'high-stress job', or 'none' for each of the n weeks. If a 'high-stress' job is chosen for $i > 1$, then 'none' should be chosen for the week $i - 1$. But, a high-stress job can be chosen in week 1. However, if a low-stress job can be chosen in a week i , then either of low-stress or high-stress job can be chosen in week $i - 1$. A low stress job gives revenue of a_i and a high-stress job gives a revenue of b_i for week i . Design an algorithm to find a plan for n consecutive weeks that maximizes the revenue.

- How do we define verification here?

- An example:**

a_i -values: 14, 7, 24, 10, 18, 8.

b_i -values: 20, 12, 30, 15, 21, 10.

→ certificate.

Schedule: high-stress, low stress,
low stress, high stress,
low stress, low stress.

Conjunctive Normal Form

- **CNF-Satisfiability:** A boolean formula

$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_4) \wedge (\bar{x}_2 \vee x_3 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_3 \vee x_4).$$

- How do we define verification of CNF-SAT?

$$c_1 = (x_1 \vee x_2 \vee x_3)$$

$$c_2 = (\bar{x}_1 \vee x_2 \vee \bar{x}_4)$$

$$c_3 = (\bar{x}_2 \vee x_3 \vee \bar{x}_4)$$

$$c_4 = (x_1 \vee \bar{x}_3 \vee x_4)$$

Clause: Disjunction of literals.

Variables: x_1, x_2, x_3, x_4

Literal: A variable either in its own form or in its complemented form.

Question: Is ϕ satisfiable?

$x_1 = \text{true}$
 $x_2 = \text{false}$

clauses.
 $x_1 = \text{false}$
 $x_2 = \text{false}$
 $x_3 = \text{false}$

$x_3 = \text{false}$
 $x_4 = \text{false}$

→ Satisfying

↓
not Satisfying

Certificate: An explicit assignment to each of the variables.

Satisfiable: If there exists an assignment that evaluates ϕ true.

Class of Problems

- Class **P**: $P = \{ L : \text{there exists an algorithm that decides } L \text{ in polynomial time} \}$.

Example: SHORTEST PATH.
LONGEST PATH in DAG, SORTING

- Class **NP**:

$NP = \{ L : \text{there exists an algorithm that can verify the problem in polynomial time} \}$

Example: 3CNFSAT (3 SAT)

INDEPENDENT SET

- Polynomial time Reducibility: Let L_1, L_2 be two problems

If there exists an algorithm A that takes an input instance x_1 of L_1 , constructs an input instance x_2 of L_2 s.t. $x_1 \in L_1$ if and only if $x_2 \in L_2$.
Then we say that $L_1 \leq_p L_2$

- Class NP-Complete:

There is a polynomial-time reduction from L_1 to L_2 .

A problem L is NP-Complete if

— $L \in \text{NP}$.

— for all $L' \in \text{NP}$, it holds that $L' \leq_p L$

maximum

- INDEPENDENT SET: Given a graph $G = (V, E)$. Find a subset $S \subseteq V(G)$ with ~~minimum~~ number of vertices such that for every $u, v \in S$, (u, v) is not an edge in G .

Decision Version: Input: G , and an integer k .

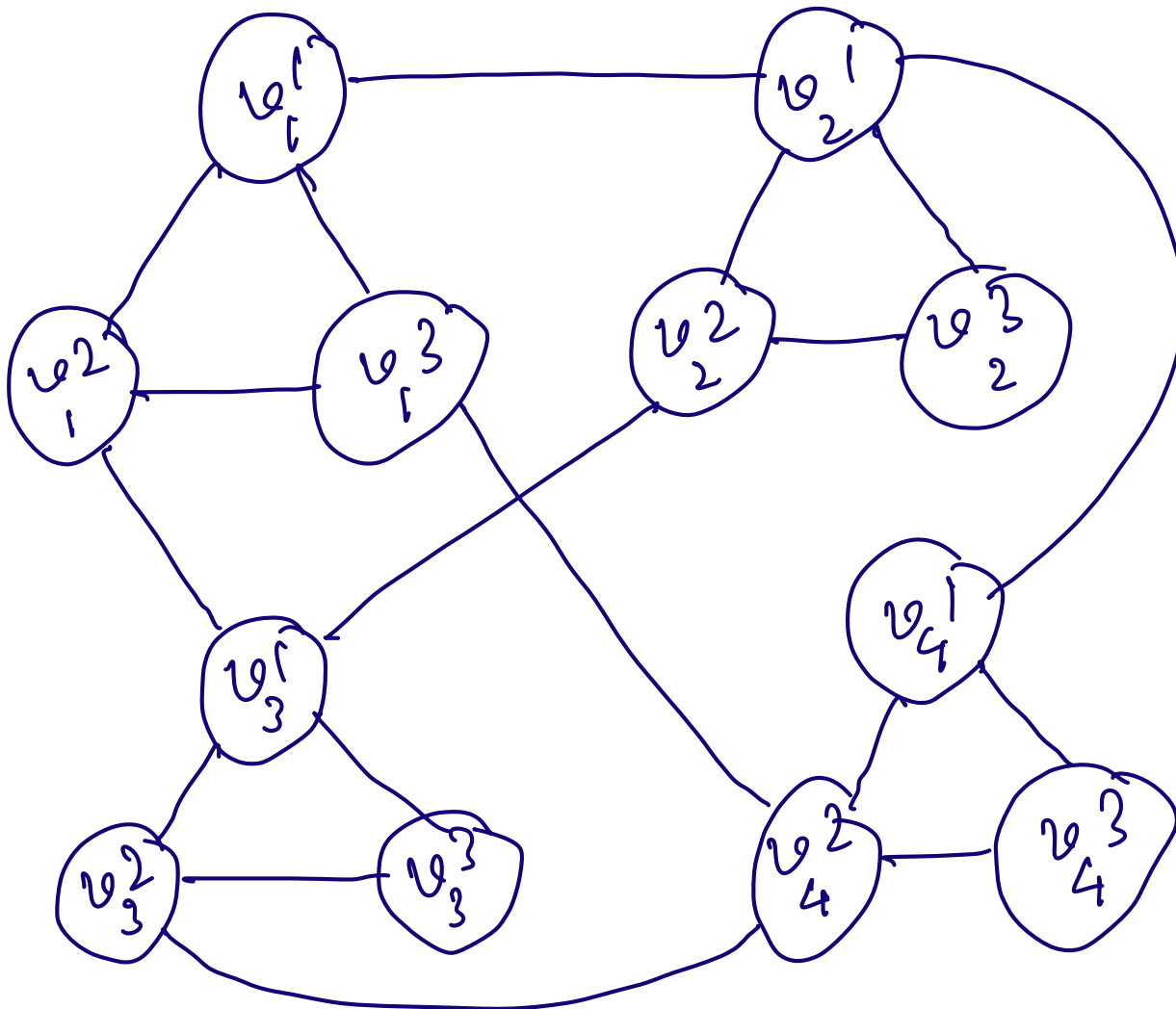
Question: Is there $S \subseteq V(G)$ s.t. for all $u, v \in S$
 (u, v) is not an edge in G and $|S| \geq k$?

Certificate: A subset $X \subseteq V(G)$.

- Reduction from 3-CNF-SAT to INDEPENDENT SET.

- **CNF-Satisfiability:** A boolean formula

$$\underbrace{(x_1 \vee x_2 \vee x_3)}_{c_1} \wedge \underbrace{(\bar{x}_1 \vee x_2 \vee \bar{x}_4)}_{c_2} \wedge \underbrace{(\bar{x}_2 \vee \bar{x}_3 \vee \bar{x}_4)}_{c_3} \wedge \underbrace{(x_1 \vee \bar{x}_3 \vee x_4)}_{c_4} = \phi$$



Outline

- ① Decision vs Search Problems - Computational Intractability
- ② Another problem in Network Flow

- Due to large-scale flooding in a region, paramedics have identified that there are n injured people and they have to be distributed across k hospitals in that region. An injured people have to be brought to a hospital that is within 30 minutes driving distance from their current location. At the same time, one does not want to overload any hospital by sending too many patients its way. They want to work out whether each injured person can be taken to a hospital such that each hospital receives at most $\lceil n/k \rceil$ people.